Forecasting Privacy-Budget Consumption in Repeated DP-SQL Workloads

Refik Can Öztaş

Hacettepe University, Ankara, Turkey
canoztas06@gmail.com

Abstract. Differentially private (DP) SQL systems add calibrated noise to each aggregate query and charge it a slice of a fixed privacy budget; once it is spent, no further query can be answered. They account for queries in isolation, yet real analytical workloads *repeat*—dashboards refresh the same tiles, alerting jobs re-poll the same counts—and an exact repeat can reuse an already-noised answer for free (DP *post-processing*). We give a closed-form model that predicts, from a workload's repetition structure alone and *before any query runs*—or, when the repetition distribution is unknown, from an early prefix—how much budget it will spend. The key quantity is the expected number of *distinct* queries, $\mathbb{E}[u_k] = \sum_i [1 - (1 - p_i)^k]$; from it we obtain the expected spend, a savings ratio, and limit, monotonicity, concentration, and utility consequences. To our knowledge this is the first such forecast computed in *closed form from public structure alone*—$\{p_i\}$, length k, budget B—with no query executed and the sensitive data never touched. We instantiate the model in a unit-tested middleware (COUNT/SUM/AVG, GROUP BY, top-k); a 4,155-trial *self-consistency* campaign matches the formula within 3% in 21/24 cells, and—as external evidence—on *Redbench*, 30 Amazon Redshift–derived traces, forecasting from only the first half of each workload cuts the prediction error by 45% over a plug-in estimate.

Keywords: Differential privacy · Privacy budget · SQL workloads · Unseen-species estimation · Caching.

1 Introduction

Aggregate statistics leak. Two COUNT queries that differ by one predicate reveal an individual's attribute by differencing; repeated, such queries reconstruct it [2]. Differential privacy (DP) [1] is the standard defense: perturb each answer with noise calibrated to its sensitivity and a per-query budget ε_q, and bound the cumulative loss by composition. The budget is a hard, non-renewable resource: once the total reaches a cap B, no further query can be answered. A custodian therefore needs to predict *how fast a workload spends its budget.*

Existing DP-SQL systems charge every query a fresh ε_q and account for them in isolation. Yet analytical workloads are highly repetitive: a monitoring dashboard re-issues the *exact same* tile on a schedule (a canonicalized exact-query repeat, served free), while a drill-down reuses a template with *different*

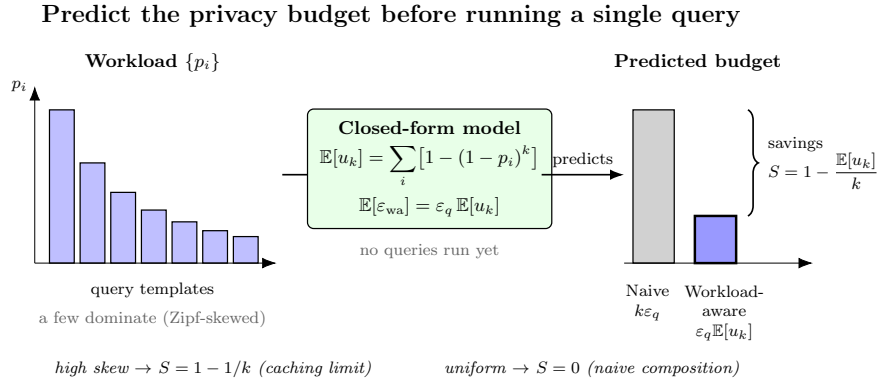

Fig. 1. Naive composition charges every one of k queries; workload-aware accounting charges only the u_k distinct query species, since exact repeats are served from cache by post-processing. The model predicts $\mathbb{E}[u_k]$, and hence the budget, before execution.

literals (a distinct query, charged afresh). Under exact-repeat caching a repeated answer is free—reusing an already-released noisy value adds no privacy cost, since any function of a DP output is itself DP (*post-processing*)—so the budget a length-k workload consumes is governed not by k but by its number of *distinct* queries u_k (Fig. 1). We ask whether u_k, and hence the budget, can be *predicted from the workload's repetition structure, in closed form, before the workload runs.* We claim no novelty for exact-repeat caching—it is a known mechanism; the contribution is *forecasting its budget impact before execution.*

Contributions.

1. **A before-execution budget forecast** (Sect. 3)—to our knowledge the first that is *closed-form* and reads only public structure $(\{p_i\}, k, B)$, never touching the data. It delivers three outputs of different strengths: the *expected* spend $\varepsilon_q\mathbb{E}[u_k]$; with the safe sizing $\varepsilon_q{=}B/m$, a *hard* completion guarantee ($u_k \le m$ in the static regime, i.e. no freshness-driven re-noising); and a *per-release* (α, β)-accuracy verdict (each release individually, not jointly). Five propositions establish it, with two limits recovering the folklore $1-1/k$ caching rule and standard per-query accounting.
2. **A measured improvement on real traces.** On 30 Redbench workloads (from Amazon Redshift logs), forecasting the full distinct-query count from only the *first half* via an unseen-species correction cuts the normalized error by 45% ($0.22 \to 0.12$) over the plug-in (Sect. 5).
3. **A middleware and validation** (Sects. 4–5)—DuckDB/PostgreSQL, 134 unit tests; the forecast matches within 3% in 21/24 cells, invariant to data scale and ε_q.

We do *not* claim to outperform deployed DP caches: against them we are *on par* on the savings number—the contribution is the forecast they do not expose.

2 Related Work

DP-SQL systems. PINQ [3] and Chorus [4] expose a fixed per-query ε_q and account in isolation. PrivateSQL [5] spends a one-time budget building DP synopses over a representative workload; it fixes that budget up front but emits no closed-form, before-execution prediction for an arbitrary repeated stream.

DP caching and proactive answering. DP caches make repeated work cheaper at runtime: Turbo [6] conserves 1.7–15.9× budget by generalizing across overlapping linear queries, and CacheDP [7] answers a workload at lower budget under an accuracy contract. Pioneer [8] reuses past noisy results but, by its authors' statement, optimizes only the *current* query and leaves future-workload effects to future work. Closest to us is the concurrent, unrefereed LAPRAS [9], also *proactive*: it precomputes an offline batch mechanism on a *predicted* query set before the stream and serves it for free—but it needs a concrete predicted set and *touches the sensitive data* ahead of time, whereas we forecast in closed form from *public* structure $(\{p_i\}, k, B)$ alone, the data never touched. These deliver the *savings* our model prices, but reactively or by precomputing on data, and without a structural forecast; our exact-repeat caching is a special case for which we claim no novelty.

Accounting, allocation, unseen species. Tight composition (RDP [12], zCDP [13]) reduces the *rate* of spend, orthogonal to our *count* of releases. *Workload-aware* mechanisms in the established sense (the matrix mechanism [16], HDMM [17]) optimize noise for a *known* batch of linear queries by running on the data, and budget *schedulers* (DPF/PrivateKube [18], Cohere [19]) treat the budget as a schedulable resource across competing consumers at runtime; neither forecasts a repeated stream's spend in closed form before execution. APEx [10] maps a per-query accuracy target to the minimum ε at runtime; multi-analyst budget sharing [11] invokes the coupon-collector problem to bound queries-per-*analyst*, a throughput limit, not a distinct-query forecast. The distinct-count side is classical occupancy/coupon-collecting [14]; predicting the not-yet-seen remainder is the unseen-species problem: Good–Toulmin [20] and its smoothed variant [21] predict new species from a prefix; this has even been brought under DP for *estimation* (INSPECTRE [22]), but for support coverage, not budget. We connect these tools to DP-*budget* forecasting.

3 The Model

The budget-spending unit ("species") is the *exact* query—a template together with its WHERE literals, i.e. the cache key. Let m be the number of distinct exact queries an analyst may issue and $\{p_i\}_{i=1}^{m}$ a distribution over them; the

analyst issues k queries i.i.d., and u_k counts the distinct queries that appear at least once.

Proposition 1 (Expected distinct queries). $\mathbb{E}[u_k] = \sum_{i=1}^{m} \big[1 - (1-p_i)^k\big]$.

Proof. With $X_i = \mathbf{1}[\text{query } i \text{ appears}]$, $\mathbb{P}[X_i{=}0] = (1-p_i)^k$; linearity gives the sum. □

In plain terms, $(1-p_i)^k$ is the chance query i never appears; one minus that is the chance it appears at least once; summing counts the distinct queries—the only ones that cost budget. *Example:* three tiles asked 70/20/10% over k=100 refreshes give $\mathbb{E}[u_k] \approx 3$: you pay for ≈ 3 queries, not 100.

Proposition 2 (Budget and savings). *Naive composition spends* $k\varepsilon_q$*, while workload-aware accounting spends* $\mathbb{E}[\varepsilon_{\text{wa}}] = \varepsilon_q \mathbb{E}[u_k]$ *in expectation—a savings ratio* $S(k) = 1 - \mathbb{E}[u_k]/k$.

Three limiting cases check the corners. *(A)* deterministic repetition (p_1=1): $S = 1 - 1/k$, the folklore caching rule. *(B)* uniform with $m \to \infty$: $\mathbb{E}[u_k] \to k$, $S \to 0$, recovering naive composition. *(C)* uniform with $k \to \infty$, m fixed: the budget saturates at $m\varepsilon_q$, so a finite-template workload cannot exhaust a budget large enough to cover all m queries.

Proposition 3 (Skew monotonicity). *For* $p_i \propto i^{-\alpha}$*,* $S(k)$ *is non-decreasing in* α *for fixed* (k, m).

Proof sketch. $\mathbb{E}[u_k] = \sum_i \phi(p_i)$ with $\phi(x) = 1 - (1-x)^k$ concave, hence $\mathbb{E}[u_k]$ is Schur-concave. For $\alpha_2 > \alpha_1$ the ratio $p_i(\alpha_2)/p_i(\alpha_1) \propto i^{-(\alpha_2 - \alpha_1)}$ is strictly decreasing in i, so the two equal-sum, sorted vectors cross exactly once, from above to below ($p_1(\alpha_2) > p_1(\alpha_1)$ is forced by the equal sums and the decreasing ratio); a single +-to-− crossing gives partial-sum dominance, i.e. $p(\alpha_2)$ majorizes $p(\alpha_1)$, weakly lowering $\mathbb{E}[u_k]$ (strictly for $k \geq 2$) and raising S (a numerical scan over $3 \leq m \leq 100$, $2 \leq k \leq 200$ corroborates). So the more a workload concentrates on a few popular queries, the more it saves.

Proposition 4 (Concentration). *With* $q_i = 1 - (1-p_i)^k$*, the occupancy indicators are negatively associated [15] (so their covariances are* ≤ 0*), giving* $\mathrm{Var}[u_k] \leq \sum_i q_i(1-q_i) \leq m/4 =: V$ *and* $\mathbb{P}[|u_k - \mathbb{E}[u_k]| \geq t] \leq V/t^2$.

Because $V = O(m)$ not $O(k)$, a single run's realized spend deviates from the forecast only on the $O(\sqrt{m})$ scale, independent of k—useful for sizing, though a variance/Chebyshev-scale statement, not a high-probability envelope.

Proposition 5 (Utility at fixed budget). *At a fixed total* ε_{tot}*, naive affords* $\varepsilon_{\text{tot}}/k$ *per query and workload-aware* $\varepsilon_{\text{tot}}/\mathbb{E}[u_k]$ *per distinct release; since* $\mathbb{E}[|\eta|] = \Delta f/\varepsilon_q$*, the per-query error ratio is* $k/\mathbb{E}[u_k]$.

So spreading the budget over the $\mathbb{E}[u_k]$ distinct queries (not all k) buys proportionally lower error—in the example, $\approx 33\times$ at equal privacy (in expectation;

a hard cap is needed when realized u_k exceeds $\mathbb{E}[u_k]$, which is why the static-regime safe sizing is $\varepsilon_q = B/m$). Beyond the mean, the per-release accuracy has a closed tail: for $\eta \sim \mathrm{Lap}(\Delta f/\varepsilon_q)$—where Δf is the sensitivity of *that release's* query (1 for COUNT, the column bound for SUM/AVG; Sect. 4), so the verdict is query-type-specific—$\mathbb{P}[|\eta| > \alpha] = e^{-\alpha\varepsilon_q/\Delta f}$, and each release meets $|\text{error}| \le \alpha$ with probability $\ge 1-\beta$ iff $\alpha \ge (\Delta f/\varepsilon_q)\ln(1/\beta)$. Under $\varepsilon_q{=}B/m$ the planner emits this *per-release* (α,β) verdict at $t{=}0$, before any query runs—reported per release, not as a joint guarantee.

4 System

A thin middleware sits between the analyst and a DuckDB/PostgreSQL backend. Each query is parsed, hashed to its exact cache key, and checked against a cache; a hit is returned free (post-processing), a miss is charged ε_q, noised with $\mathrm{Lap}(\Delta f/\varepsilon_q)$, and cached. Sensitivities are $\Delta f = 1$ for COUNT and $\Delta f = B_c$ for SUM/AVG, where the public bound B_c is enforced *by construction*: the executed SQL clamps each summed value to $[0, B_c]$ (NULLs preserved), so out-of-domain data cannot break the guarantee. A GROUP BY costs ε_q (parallel composition); top-k is post-processing of one noised histogram (order/limit applied on the noised values). The parser enforces a strict positive whitelist—window/arithmetic-wrapped aggregates, self-union CTEs, set operations, derived tables, and OFFSET are rejected before any budget is spent.

The model is also prescriptive: a *predictive allocator* estimates $\hat{U} \approx \mathbb{E}[u_k]$ online from the empirical query distribution and sets $\varepsilon_q = B/\hat{U}$, capping spend at B. A *temporal* extension prices freshness (staleness τ, update rate λ) into a first-order planning estimate, not a proven bound; note that re-noising charges fresh budget, so the static $u_k \le m$ cap—and with it the hard completion guarantee—does not extend to the temporal setting.

Scope boundary. The GROUP BY / top-k guarantees assume a *public, fully enumerated* key domain (absent groups released as noised zeros). Private key *discovery* (a stability threshold) is future work; the prototype returns present keys only, which we state plainly rather than hide.

5 Evaluation

Setup. A 4,155-trial campaign over Zipf workloads (Adult age-bands and TPC-H templates). The main grid ($m{=}7$, $\varepsilon_q{=}1$) crosses Zipf $\alpha \in \{0, 0.5, 1, 1.5, 2, 3\}$ with $k \in \{10, 25, 50, 100\}$ at 30 trials per cell; the savings workloads use $m \in \{1, 3, 5, 7\}$ plus an all-unique drill-down; separate sweeps cover $\varepsilon_q \in \{0.1, 0.5, 1, 2\}$ and k up to 500, on two data scales (TPC-H SF=1/10).

Forecast accuracy. The closed form matches the empirical mean within $< 3\%$ in 21/24 main-grid cells (the three misses are the smallest-k cells, Zipf $\alpha{\ge}2$ at $k{\le}25$, worst 8.6%, where a 30-trial mean is noisiest); it is invariant to data scale (6M

Table 1. Budget saved vs. naive composition (k=100, total ε=100, 30 trials). The rows vary the distinct-query *pool size* m; by k=100 the distinct count u_k saturates near m, which the model predicts in closed form. (At fixed m, higher skew lowers u_k further; Prop. 3.)

Workload	naive ε	workload-aware ε	saved
Repetitive (1 query)	100	1.0	100×
TPC-H returnflag (m=3)	100	3.0	33×
TPC-H priority (m=5)	100	5.0	20×
Uniform / Zipf pool (m=7)	100	7.0	14.3×
Drill-down (all unique)	100	100.0	1× (no win)

vs 60M rows agree within 0.034 in $\mathbb{E}[u_k]$, ≈0.7% of the m=5 pool tested) and to ε_q (it cancels), and is *exact* on a constructed mixed workload. This grid is a *self-consistency* check—trials are drawn from the assumed distribution—confirming the implementation matches the model; the deployment-sizing evidence is the Redbench prefix forecast below.

Savings and the no-win case. Workload-aware accounting answers the same 100 queries while spending only $\varepsilon_q u_k$ (Table 1): 100× less budget on a repetitive workload, 14–33× on parametric/uniform pools (where u_k saturates at m), and *exactly* 1× on a genuine drill-down—which the model correctly predicts as $S(k)$=0 rather than overclaiming.

Real traces (headline). On Redbench [23]—30 workloads synthesized from Amazon Redshift's Redset production traces—the exact-query repetition rate *is* our $S(k)$. The empirical savings $1 - u_k/k$ spans 0–98% (mean 53%) with fitted Zipf α from 0 to ≈3 (mean 0.84), a range our synthetic sweep ($\alpha \in \{0, \ldots, 3\}$) covers. Forecasting the full distinct-query count from only the *first half* of each timeline, a smoothed Good–Toulmin estimator cuts the plug-in's error by 45% (0.22→0.12 in u_k/k). This—not the in-sample fit—is the genuine ahead-of-time forecast of the unexecuted remainder, confirmed on production-derived data. Redbench is *not* a DP benchmark; we use only its repetition structure to externally validate the occupancy forecast.

Calibration and allocation. Two attacks confirm the per-release noise is correctly calibrated: a single-query membership-inference AUC stays at or below the $e^\varepsilon/(1+e^\varepsilon)$ optimal-*accuracy* reference (within one percentage point for $\varepsilon \le 1$), and a differencing reconstruction error scales as $1.5/\varepsilon$. For allocation, the safe ε_q=B/m sizing answers 100% of queries at fresh-release MAE 2.0 versus an ε-greedy bandit's 3.6—it *reaches* the bandit's best operating point at zero exploration cost (in the $m \le k/4$ regime tested). Against reactive caches we cite their own figures (Turbo 1.7–15.9×; ours 14–33×): the same order of magnitude—though the metrics differ (Turbo *generalizes across overlapping* queries, we reuse only *exact* repeats), so this is a ballpark placement, not a controlled comparison, and we claim *parity*, not superiority.

6 Conclusion

A closed-form model parameterized by $(\{p_i\}, k, B)$ predicts expected DP-budget consumption in repeated aggregate SQL workloads—the expected spend, a hard static-regime completion guarantee, and a per-release accuracy verdict—from public structure alone, before any query runs and without touching the data; the realized spend concentrates within $O(\sqrt{m})$ of the forecast (Prop. 4). We are explicit about scope: the synthetic grid is a self-consistency check (the empirical u_k is drawn from the very distribution the formula integrates), the temporal extension is first-order, the GROUP BY guarantee assumes a public key domain, and savings genuinely vanish on workloads that never repeat—which the model predicts rather than hides. The external evidence is the Redbench prefix forecast: a 45% error reduction on production-derived traces the model never saw. Code, the 134-test middleware, and an interactive demo are public.[1]

References

1. C. Dwork, F. McSherry, K. Nissim, A. Smith. Calibrating Noise to Sensitivity in Private Data Analysis. TCC, 2006.
2. I. Dinur, K. Nissim. Revealing Information While Preserving Privacy. PODS, 2003.
3. F. McSherry. Privacy Integrated Queries (PINQ). SIGMOD, 2009.
4. N. Johnson, J. P. Near, J. M. Hellerstein, D. Song. Chorus: a Programming Framework for Building Scalable Differential Privacy Mechanisms. IEEE EuroS&P, 2020.
5. I. Kotsogiannis, Y. Tao, X. He, M. Fanaeepour, A. Machanavajjhala, M. Hay, G. Miklau. PrivateSQL: A Differentially Private SQL Query Engine. PVLDB 12(11), 2019.
6. K. Kostopoulou, P. Tholoniat, A. Cidon, R. Geambasu, M. Lécuyer. Turbo: Effective Caching in Differentially-Private Databases. SOSP, 2023.
7. M. Mazmudar, T. Humphries, J. Liu, M. Rafuse, X. He. Cache Me If You Can: Accuracy-Aware Inference Engine for Differentially Private Data Exploration. PVLDB 16(4), 2022.
8. S. Peng, Y. Yang, Z. Zhang, M. Winslett, Y. Yu. Query Optimization for Differentially Private Data Management Systems (Pioneer). ICDE, 2013.
9. P. Mundra, A. Sealfon, Z. Sun, Q. C. Liu. LAPRAS: Learning-Augmented Private Answering for Linear Query Streams. arXiv:2605.01960, 2026.
10. C. Ge, X. He, I. F. Ilyas, A. Machanavajjhala. APEx: Accuracy-Aware Differentially Private Data Exploration. SIGMOD, 2019.
11. D. Pujol, A. Sun, B. Fain, A. Machanavajjhala. Multi-Analyst Differential Privacy for Online Query Answering. PVLDB 16(4), 2022.
12. I. Mironov. Rényi Differential Privacy. CSF, 2017.
13. M. Bun, T. Steinke. Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds. TCC, 2016.
14. P. Flajolet, D. Gardy, L. Thimonier. Birthday Paradox, Coupon Collectors, Caching Algorithms and Self-Organizing Search. Discrete Applied Math 39(3), 1992.

[1] `https://github.com/canoztas/cmp653-project`

15. K. Joag-Dev, F. Proschan. Negative Association of Random Variables, with Applications. Ann. Statist. 11(1), 1983.
16. C. Li, G. Miklau, M. Hay, A. McGregor, V. Rastogi. The Matrix Mechanism: Optimizing Linear Counting Queries under Differential Privacy. VLDB Journal 24(6), 2015.
17. R. McKenna, G. Miklau, M. Hay, A. Machanavajjhala. Optimizing Error of High-Dimensional Statistical Queries under Differential Privacy. PVLDB 11(10), 2018.
18. T. Luo, M. Pan, P. Tholoniat, A. Cidon, R. Geambasu, M. Lécuyer. Privacy Budget Scheduling. OSDI, 2021.
19. N. Küchler, E. Opel, H. Lycklama, A. Viand, A. Hithnawi. Cohere: Managing Differential Privacy in Large Scale Systems. IEEE S&P, 2024.
20. I. J. Good, G. H. Toulmin. The Number of New Species, and the Increase in Population Coverage, when a Sample is Increased. Biometrika 43, 1956.
21. A. Orlitsky, A. T. Suresh, Y. Wu. Optimal Prediction of the Number of Unseen Species. PNAS 113(47), 2016.
22. J. Acharya, G. Kamath, Z. Sun, H. Zhang. INSPECTRE: Privately Estimating the Unseen. ICML, 2018.
23. S. Krid, M. Stoian, A. Kipf. Redbench: A Benchmark Reflecting Real Workloads. aiDM @ SIGMOD, 2025. arXiv:2506.12488.